NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

ASSIGNMENT FOR COMPUTER GRAPHICS

notes.arpanadhikari7.com.np

ASSIGNMENT 4 (3D)

Submitted by:
Name: Arpan Adhikari
Roll No.: 231309
BE-CE (3rd SEM)

Submitted to:
Saroj Shakya

Available on notes.arpanadhikari7.com.np

Name : Arpan Adhikari
Roll No. : 231309
BECE (Day)

Assignment 4 (CG)

Q1) Explain the issues related to 3D viewing that make 3D viewing more complex than simple 2D viewing.

→

3D viewing is more complex than 2D viewing because it involves additional challenges related to depth perception, realism and rendering. Here are key issues:

1. Depth Perception
3D viewing requires simulating depth, unlike 2D, which is flat. Techniques like parallax and stereoscopic vision are used.

2. Projection Complexity
3D objects in (x,y,z) need to be projected into 2D, requiring perspective or orthographic projections.

notes.arpanadhikari7.com.np

3. Camera Handling
In 3D, the camera has more freedom (position, angle, field of view) compared to 2D, where its static.

4. Occlusion
3D requires handling object overlap and visibility using z-buffering or painter's algorithm.

5. Lighting and shadows
Realistic 3D lighting requires complex calculations for reflections, shadows and shading.

Available on notes.arpanadhikari7.com.np

6. Rendering Performance
3D rendering involves intensive computations for transformations, textures and lighting, unlike

7. Perspective Distortion
3D objects must appear closer or farther based on their position, adding complexity to rendering.

8. Interaction Complexity
Interacting with 3D objects involves managing orientation, depth and positioning, unlike 2D click-based interaction.

9. Parallax and stereo vision
3D simulates how eyes perceive depth, requiring stereoscopic techniques or VR, which 2D doesn't need.

10. Realistic Animation
Animating 3D objects involves movement, rotation and physics in a 3D space, making it more complex than 2D.

notes.arpanadhikari7.com.np

Q2) How does 2D clipping differ from 3D clipping?

→

The differences are :

1. Dimensions
2D : Operates in (x,y)
3D : Works in (x,y,z)

2. Clipping Boundaries
2D : Uses a rectangular or polygonal viewport.
3D : clips objects within a 3D viewing frustum (near & far planes)

Available on notes.arpanadhikari7.com.np

3. Clipping Regions
2D : Objects clipped inside a 2D window.
3D : Objects clipped within a 3D volume projected to 2D.

4. Objects clipped
2D : Points, lines, polygons
3D : Planes, surfaces, volumes

5. complexity
2D : simpler, fewer coordinates
3D : more complex due to depth.

6. Algorithms
2D : cohen-sutherland,
3D : Extends 2D algorithms to 3D or uses specialized techniques like Sutherland-Hodgman in 3D.

Q3) Derive the scaling transformation matrix of a point $P = (x, y, z)$ with respect to a fixed point (x_f, y_f, z_f)

→

notes.arpanadhikari7.com.np

Scaling changes size of an object and repositions the object relative to the coordinate origin.

After applying scaling to a point (x, y, z),

$$x' = x * S_x$$
$$y' = y * S_y$$
$$z' = z * S_z$$

So, matrix expression for scaling transformation of a position $P = (x, y, z)$ relative to coordinate origin can be written as,

Available on notes.arpanadhikari7.com.np

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

For scaling with respect to fixed point (x_f, y_f, z_f), it involves the following steps:

(i) Translate fixed point to the origin.
(ii) Scale object relative to coordinate origin.
(iii) Translate fixed point back to its original position.

So, $C_m = T_{(x_f, y_f, z_f)} \cdot S_{(s_x, s_y, s_z)} \cdot T_{(-x_f, -y_f, -z_f)}$

$$= \begin{pmatrix} 1 & 0 & 0 & x_f \\ 0 & 1 & 0 & y_f \\ 0 & 0 & 1 & z_f \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & -z_f \\ 0 & 0 & 0 & 1 \end{pmatrix}$$ Ans

Q4) Derive a transformation matrix for producing orthographic and oblique parallel projection on the x_v, y_v plane.

→

Orthographic parallel projection:

When projection is perpendicular to view plane, we have orthographic parallel projection.

While doing orthographic parallel projection, the x_v, y_v plane corresponds to $z = 0$ plane.

Here, $x' = x$

$y' = y$

$z = 0$

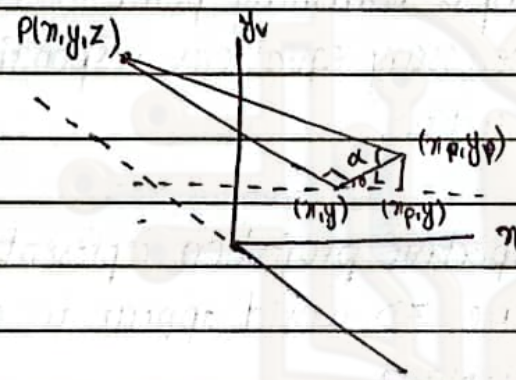
Available on notes.arpanadhikari7.com.np

In matrix form,

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Oblique Parallel Projection:

In oblique projection, the projection lines are not perpendicular to the projection plane but are instead skewed at an oblique angle. Oblique projection line from $P(x,y,z)$ to (x_p, y_p) makes an angle 'α' with the line on projection plane.



The leng line of length 'L' is at an angle θ with the horizontal direction in the projection plane.

Now, to Expressing x_p and y_p in terms of x, y, L, θ,

$$x_p = x + L\cos\theta$$
$$y_p = y + L\sin\theta \quad \text{—— (1)}$$

The value of L depends on α and z-co-ordinate of point being projected,

Here, $\tan\alpha = \frac{z}{L}$

or, $L = zL_1$ where, $L_1 = \frac{1}{\tan\alpha}$

Available on notes.arpanadhikari7.com.np

Hence, the equations (1) are,

$$x_p = x + zL_1\cos\theta$$
$$y_p = y + zL_1\sin\theta$$

The transformation matrix for producing parallel projection onto the x_v, y_v plane can be written as,

$$M_{parallel} = \begin{pmatrix} 1 & 0 & L_1\cos\theta & 0 \\ 0 & 1 & L_1\sin\theta & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Q5) What is the significance of a vanishing point in Perspective Projection? Explain the necessary equations responsible for producing perspective projection.

→

A vanishing point in perspective projection represents the point where parallel lines in the 3D world appear to converge when projected onto a 2D view plane.

notes.arpanadhikari7.com.np

Key significance:

(i) Depth cue: Vanishing points create the illusion of depth, making 2D representations appear 3D.

(ii) Realistic visualization: They help mimic how the human eye perceives distant objects as smaller and converging.

(iii) Projection consistency: For lines parallel to a specific axis, the vanishing point determines the perspective distortion.

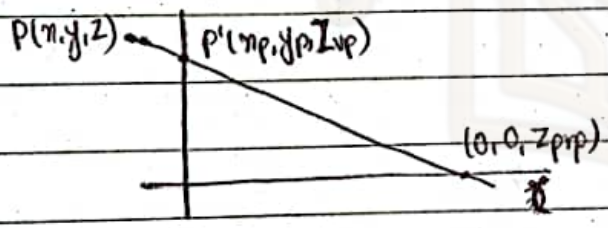
Equations: →

Available on notes.arpanadhikari7.com.np

Suppose the perspective reference point is set at position z_{prp} along z-axis, the view plane is at z_{vp}.
Expressing the equations, describing the coordinate positions along this perspective projection line in a parametric form,

$$x' = x - x.u$$
$$y' = y - y.u$$
$$z' = z - (z - z_{prp}).u$$

Parameter 'u' takes values from 0 to 1, coordinate position $P'(x', y', z')$ represents any point along projection line. When $u = 0$, we are at position $P(x, y, z)$. At other end of line, $u = 1$, we have the projection reference point coordinates $(0, 0, z_{prp})$.



notes.arpanadhikari7.com.np

On the view plane, $z' = z_{vp}$.

$$\therefore z_{vp} = z - zu + z_{prp}u$$
$$\text{or, } z_{vp} - z = z_{prp}u - zu$$
$$\therefore u = \frac{z_{vp} - z}{z_{prp} - z}$$

Available on notes.arpanadhikari7.com.np

Substituting value of 'u', we can find the values of x_p, y_p.

$$\Rightarrow x_p = x - x\left(\frac{z_{vp} - z}{z_{prp} - z}\right)$$

$$\therefore x_p = \frac{x(z_{prp} - z_{vp})}{z_{prp} - z} = \frac{x \cdot d_p}{z_{prp} - z}$$

$$\Rightarrow y_p = y - y\left(\frac{z_{vp} - z}{z_{prp} - z}\right)$$

$$\therefore y_p = \frac{y(z_{prp} - z_{vp})}{z_{prp} - z} = y \cdot \frac{d_p}{z_{prp} - z}$$

where, $d_p = z_{prp} - z_{vp}$, the distance of viewplane from the projection reference point.

Using 3D homogeneous coordinate representation, we can write perspective projection transformation in matrix form as,

$$\begin{pmatrix} x_h \\ y_h \\ z_h \\ h \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -z_{vp}/d_p & z_{vp}(z_{prp}/d_p) \\ 0 & 0 & -1/d_p & z_{prp}/d_p \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

notes.arpanadhikari7.com.np

where, homogeneous factor is: $h = \frac{z_{prp} - z}{d_p}$

and the projection coordinates on the view plane are calculated from the homogeneous coordinates as:

$$x_p = \frac{x_h}{h} \qquad y_p = \frac{y_h}{h}$$

Available on notes.arpanadhikari7.com.np

Q6) Find the perspective projection of a tetrahedron A(3,4,0) B(1,0,4) C(2,0,5) , D(4,0,3) onto a projection plane situated at 0. The center of projection should be located at -5.

Soln

Given points,

A(3,4,0) , B(1,0,4) , C(2,0,5) , D(4,0,3)

For perspective projection, COP = Projection reference point = (0,0,-5)

i.e. $z_{prp} = -5$

To project to a plane at 0 (i.e. $z_{vp} = 0$),

For A(3,4,0) i.e(x,y,z)

$$x_p = x\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)}$$

$$= 3\left(\frac{-5-0}{-5-0}\right) = \frac{3 \times -5}{-5} = 3$$

$$y_p = y\frac{(z_{prp} - z_{vp})}{z_{prp} - z} = 4 \times \left(\frac{-5-0}{-5-0}\right) = 4 \quad (3,4,0)$$

notes.arpanadhikari7.com.np

For B(1,0,4),

$$x_p = x\left(\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)}\right)$$

$$= 1\left(\frac{-5-0}{-5-4}\right) \quad \therefore x_p = 0.56$$

$$y_p = y\left(\frac{z_{prp} - z_{vp}}{z_{prp} - z}\right) \quad \therefore y_p = 0 \quad (0.56, 0, 0)$$

For C(2,0,5),

$$x_p = x\left(\frac{z_{prp} - z_{vp}}{z_{prp} - z}\right) = 2\left(\frac{-5-0}{-5-5}\right) = 1$$

Available on notes.arpanadhikari7.com.np

$$y_p = y_x \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right)$$

$= 0 \times$ $\therefore y_p = 0$ (1,0,0)

For D(4,0,3),

$$x_p = x \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right)$$

$$= 4 \left(\frac{-5-0}{-5-3} \right) = 2.5$$

$$y_p = y \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right)$$

$\therefore y_p = 0$

Hence, points after projection are A'(3,4,0), B'(0.56,0,0), C'(1,0,0), D'(2.5,0,0)

Q7) What are different types of parallel and perspective projection?

→

Projection techniques are used in computer graphics and technical drawing to represent 3D objects on a 2D plane. They can be broadly classified as parallel projection & ~~perpers~~ perspective projection.

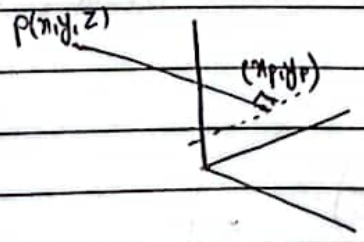
(i) Parallel Projection

notes.arpanadhikari7.com.np

In parallel, the projection lines are parallel and donot converge to a single point. The lines are perpendicular to projection plane.

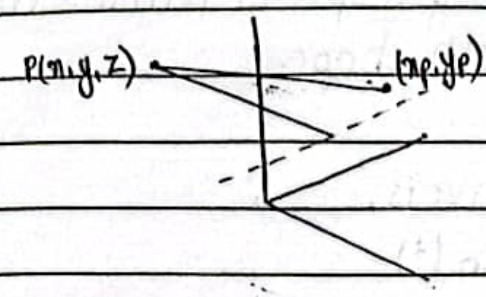
A. Orthographic Parallel Projection

The projection lines are perpendicular to the projection plane. Used in engineering drawings, CAD, and architectural plans.



Available on notes.arpanadhikari7.com.np

B. Oblique Parallel Projection
The projection lines are not perpendicular but at an angle to the projection plane.
Used for quick sketches and illustrations.



(ii) Perspective Projection
Unlike parallel projection, perspective projection makes distant objects appear smaller, creating sense of depth and realism.
It is widely used in art, animation, 3D modeling etc.

Types of Perspective:

A. One-Point Perspective
- One vanishing point on the horizon.
- Suitable for railroads, hallways or roads viewed straight ahead.

B. Two-Point Perspective
- Two vanishing point on the horizon.
- common in architectural sketches and 3D modeling

C. Three-Point Perspective
- Three vanishing point, including one above or below.
- Used for skyscrapers, tall buildings, or extreme angles.

Available on notes.arpanadhikari7.com.np

Q8) What are the issues regarding drawing a spline using Bezier blending function? Explain with the necessary equations and properties.

→

A Bezier curve is calculated using a special formula that blends control points to create a smooth shape.

The basic equation for Bezier curve is,

$$B(t) = \sum_{i=0}^{n} B_i N_{i,n}(t)$$

Here, B_i are control points and $N_{i,n}(t)$ are blending functions.

Problems making spline with Bezier curves:

1. Hard to make a smooth connection
When connecting multiple Bezier curves to make a spline, the joints may not be smooth. The curve might have sharp corners instead of a ~~sme~~ seamless flow.

2. Changing one control Point Affects the whole curve
In Bezier curves, if we move one control point, the entire curve shape changes. This makes it hard to adjust only a small part of the spline.

3 More control points make it harder to compute
A simple bezier curve works well with few control points. If we add too many control points, the formula becomes complicated and curves can "wiggle" too much instead of forming smooth shape.

4 Choosing Good control Points is difficult
We need to carefully place the control points to get the shape

Available on notes.arpanadhikari7.com.np

we want. If control points are unevenly space, the curve can look strange.

Q9) Explain the role of Blending function, convex Hull and control Points used in Bezier curve.

→

Blending Function

The blending function determines how much influence each control point has on the curve at any given point.
It is defined using Bernstein polynomials:

$$N_{i,n}(t) = \binom{n}{i} (1-t)^{n-i} t^i$$

where, n is the degree of Bezier curve.
i is index of control point
t is parameter that moves from 0 to 1 along curve
$\binom{n}{i}$ is binomial coefficient

Roles:

1. It assigns different weights to each control point based on t.
2. The blending function smoothly transitions between control points
3. By modifying control points, the blending function adjust final shape.

Convex Hull Property:

1. Bounding Area
Helps in quick calculations for collision detection & clipping

2. Prevents unwanted collisions
Ensures the curve doesn't deviate wildly.

3. Maintains stability
The curve shape remains predictable.

Available on notes.arpanadhikari7.com.np

Control Points:

1. Shape Defining
Moving a control point alters the curve.

2. Degree control
The number of control points determines the curve's degree.

3. Local vs Global Effect
Adjusting one point effects the entire curve.

Q10) Derive matrix used for forming a quartic spline using the method proposed by Bezier.

→

A Bezier curve of degree n is defined as,

$$P(u) = \sum_{i=0}^{n} P_i B_{i,n}(u) \quad \text{where } 0 \le u \le 1$$

where, $B_{i,n}(u) = C(n,i) u^i (1-u)^{n-i}$

$$C(n,i) = \frac{n!}{i!(n-i)!}$$

$$\text{so, } B_{i,n}(u) = \frac{n!}{i!(n-i)!} u^i (1-u)^{n-i}$$

For quartic spline, $n = 4$,

$$P(u) = P_0 B_{0,4}(u) + P_1 B_{1,4}(u) + P_2 B_{2,4}(u) + P_3 B_{3,4}(u) + P_4 B_{4,4}(u)$$

$$\text{so, } B_{0,4}(u) = \frac{4!}{0!\,4!} u^0 (1-u)^4 = (1-u)^4$$

Available on notes.arpanadhikari7.com.np

$$B_{1,4}(u) = \frac{4!}{1!3!} u^1 (1-u)^3$$

$$= 4u(1-u)^3$$

$$B_{2,4}(u) = \frac{4!}{2!2!} u^2 (1-u)^2 = 6u^2(1-u)^2$$

$$B_{3,4}(u) = \frac{4!}{3!1!} u^3 (1-u)^1 = 4u^3(1-u)$$

$$B_{4,4}(u) = \frac{4!}{4!0!} u^4 (1-u)^0 = u^4$$

So, final blending function is,

$$P(u) = P_0(1-u)^4 + P_1 4u(1-u)^3 + 6u^2(1-u)^2 P_2 + 4u^3(1-u)P_3 + u^4 P_4$$

In matrix form,

$$P(u) = \begin{bmatrix} (1-u)^4 & 4u(1-u)^3 & 6u^2(1-u)^2 & 4u^3(1-u) & u^4 \end{bmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \\ P_4 \end{pmatrix}$$

$$= \begin{bmatrix} (1+u^4-4u+6u^2-4u^3) & (4u-12u^2+12u^3-4u^4) & (6u^2-12u^3+6u^4) \rightarrow \\ \rightarrow (4u^3-4u^4) & u^4 \end{bmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \\ P_4 \end{pmatrix}$$

$$\left[\because (a-b)^4 = a^4+b^4-4a^3b+6a^2b^2-4ab^3\right]$$

$$= \begin{pmatrix} u^4 & u^3 & u^2 & u & 1 \end{pmatrix} \begin{pmatrix} 1 & -4 & 6 & -4 & 1 \\ -4 & 12 & -12 & 4 & 0 \\ 6 & -12 & 6 & 0 & 0 \\ -4 & 4 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \\ P_4 \end{pmatrix}$$

Ans

Available on notes.arpanadhikari7.com.np

Q11) How is non-planar surface formed using spline? Explain Bezier surface with equations. Explan its properties.

→

A non-planar surface is formed using splines by extending the concept of spline curves to two dimensions. Instead of defining a curve with control points, we define a surface patch using a grid of control points.

A Bezier surface is a smooth surface defined over a rectangular domain using a grid of control points. It is constructed as a tensor product of Bezier curves.

A Bezier surface of degree (m,n) is,

$$P(u,v) = \sum_{j=0}^{m} \sum_{k=0}^{n} P_{j,k} \, BEZ_{j,m}(u) \, BEZ_{k,n}(v)$$

As in the case of Bezier curves the $P_{j,k}$ define the control vertices and the $BEZ_{j,m}(u)$ and $B_{k,n}(v)$ are the Bernstein blending function in the u and v directions.

notes.arpanadhikari7.com.np

The Bezier functions specify the weighting of the particular knot. They are the Beinstein coefficients. The definition of the Bezier function is,

$$BEZ_{j,m}(u) = C(m,j) \, u^j (1-u)^{m-j}$$
$$BEZ_{k,n}(v) = C(n,k) \, v^k (1-v)^{n-k}$$

where,

$$C(m,j) = \frac{m!}{j!(m-j)!} \qquad C(n,k) = \frac{n!}{k!(n-k)!}$$

Available on notes.arpanadhikari7.com.np

Properties of Bezier surfaces:

- The surface takes the general shape of the control points.
- The surface is contained within the convex hull of control points.
- The corners of the surface and the corner control vertices are coincident.

Q15) Explain 3D windowing process (window to viewport mapping).

→

$$C_{nn} = T_{x_{wmin}, y_{wmin}, z_{wmin}} \; S_{s_x, s_y, s_z} \; T_{-x_{wmin}, -y_{wmin}, -z_{wmin}}$$

$$= \begin{bmatrix} 1 & 0 & 0 & x_{wmin} \\ 0 & 1 & 0 & y_{wmin} \\ 0 & 0 & 1 & z_{wmin} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_{wmin} \\ 0 & 1 & 0 & -y_{wmin} \\ 0 & 0 & 1 & -z_{wmin} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where, $s_x = \dfrac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}$

$s_y = \dfrac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$

$s_z = \dfrac{z_{vmax} - z_{vmin}}{z_{wmax} - z_{wmin}}$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np